

Differentiable shortest-path computation on graphs with cycles: relaxation versus perturbation

Many alignment and structured-prediction problems reduce to shortest-path computations on a graph, yet using such solvers as differentiable layers requires gradients through a piecewise-constant optimum. The two standard remedies (smoothing and stochastic perturbation) both assume an acyclic graph; once cycles appear, smoothing can make the recursion ill-posed while perturbation pays an unknown variance cost. The internship maps that trade-off and, building on it, aims to deliver a practical differentiable shortest-path solver that works on graphs with cycles.

§1. Organization

- **Start date:** As soon as possible (flexible).
- **Localization:** IRISA, Rennes.
- **Supervision:** Romain Tavenard (Prof., Université Rennes 2, MALT Inria team, IRISA, Rennes).

§2. Context

A wide range of structured-prediction and sequence-alignment problems reduce to shortest-path computations on a graph. Dynamic Time Warping, for instance, is a shortest path on a grid graph, and many segmentation, parsing, and ranking tasks share the same combinatorial backbone. To embed such solvers inside end-to-end trainable pipelines, one must differentiate through them. This is non-trivial: the optimal path is a piecewise-constant function of the edge weights, so the exact gradient is zero almost everywhere and useless for learning.

Two paradigms produce informative gradients. *Continuous relaxation* replaces the hard min by a smooth surrogate, as in differentiable dynamic programming (Mensch & Blondel, 2018) and Soft-DTW (Cuturi & Blondel, 2017), giving deterministic but biased gradients. *Stochastic perturbation* (Berthet et al., 2020) randomizes the solver’s inputs and averages, giving unbiased but higher-variance estimators; the black-box differentiation of combinatorial solvers (Vlastelica et al., 2020) is closely related but deterministic, recovering a gradient surrogate from a single additional solver call along the loss direction. Both treat the solver as an oracle. These tools are now textbook material (Blondel & Roulet, 2024), which also frames a cyclic computation as a *while loop* or fixed point and differentiates through it via the implicit function theorem.

§3. Scientific challenge

Cycles break the relaxation paradigm in a precise way. With a *hard* min, a positive-weight self-loop or cycle is harmless, it is simply never traversed by an optimal path. Under a *smoothed* min, however, the soft policy assigns non-zero probability to re-entering a cycle. Tellingly, the existing differentiable all-pairs method DataSP (Aviles-Rivero et al., 2024) does not solve this: it constrains all edge costs to be strictly positive and forbids self-loops, so that no cycle is ever profitable, and yet its smoothed Floyd–Warshall still generates cycle-containing paths, which the authors explicitly identify as a *bias* and remove by hand. Perturbation-based estimators sidestep the recursion entirely but it is expected that their variance scales with cycle density, since low-cost cycles introduce additional paths with non-zero probability. This internship asks: **when a shortest-path graph contains cycles, under what conditions does each differentiable paradigm remain well-posed, how do their bias, variance, and cost trade off, and can the two be combined into a single usable solver?**

§4. Goals

The end goal is a practical differentiable shortest-path solver for cyclic graphs; the analysis below is the means to design it on solid ground. Concretely, the candidate will:

1. characterize the convergence regime of smoothed shortest-path recursions on cyclic graphs as a function of smoothing temperature and cycle weights, identifying when the fixed point exists and is differentiable;
2. implement perturbation-based and black-box estimators for the same setting and quantify how their bias and variance scale with cycle density;
3. compare the paradigms on controlled benchmarks (isolated self-loops, single large cycles, varying cycle density, near-degenerate weights) along gradient bias, variance, runtime, and downstream alignment quality;
4. turn these findings into a concrete algorithm and validate it on a small end-to-end task.

Candidates should possess:

- good Python coding skills and familiarity with an autodiff framework (PyTorch or JAX),
- knowledge of basic graph algorithms (shortest paths, dynamic programming),
- comfort with numerical optimization,
- ability to communicate in English in a scientific context.

Funding for a Ph.D. has already been secured: upon a successful internship, the candidate will be offered the opportunity to continue this work as a fully funded doctoral thesis, turning the internship’s prototype into a general differentiable solver. A successful candidate will develop or reinforce skills key to that continuation, namely:

- a practical and theoretical understanding of differentiable optimization,
- the ability to move from analysis to a working, validated algorithm,
- experience implementing and benchmarking combinatorial solvers as differentiable layers.

§5. Bibliography

- Aviles-Rivero, A. I., et al. (2024). *DataSP: A Differentiable All-to-All Shortest Path Algorithm for Learning Costs and Predicting Paths with Context*. arXiv:2405.04923.
- Berthet, Q., Blondel, M., Teboul, O., Cuturi, M., Vert, J.-P., & Bach, F. (2020). *Learning with Differentiable Perturbed Optimizers*. NeurIPS.
- Blondel, M., & Roulet, V. (2024). *The Elements of Differentiable Programming*. arXiv:2403.14606.
- Cuturi, M., & Blondel, M. (2017). *Soft-DTW: A Differentiable Loss Function for Time Series*. ICML.
- Mensch, A., & Blondel, M. (2018). *Differentiable Dynamic Programming for Structured Prediction and Attention*. ICML.
- Petersen, F., Borgelt, C., Sutter, T., Kuehne, H., Deussen, O., & Ermon, S. (2024). *Newton Losses: Using Curvature Information for Learning with Differentiable Algorithms*. NeurIPS.
- Vlastelica, M., Paulus, A., Musil, V., Martius, G., & Rolínek, M. (2020). *Differentiation of Blackbox Combinatorial Solvers*. ICLR.